

Kungfu Crashout - Team11

Kungfu Crashout - Team11 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *Kungfu Crashout* is a **mobile multiplayer fighting game where the environment is the weapon** — players pick up items from the background and use them against opponents. Control is **screen-based gesture input with no on-screen buttons**, a simple scheme that's easy to pick up. Gameplay is quick 1-on-1 battles in a wide side-view stage with detailed, interactable backgrounds; matches last a few minutes. A roster of characters each have unique fighting styles, players combining the environment with their martial-arts skills to win. Named load-bearing C++: a **native Android** build; a **C++ game loop holding a stable 60 fps with 2 characters + multiple physics objects + particles**, running **deterministically; peer-to-peer PvP; AI opponents**; and **interactable stage objects**. Named technical features: **gestures as player actions** (movement, attacks, interactions); **environmental interactions** (pick up + throw objects); **different enemy AIs** (difficulty / character-based); and **multiplayer PvP**. Tools: a **custom editor** to view/debug the game — **hurtbox/hitbox debugging + frame checking** of movement and animation, a **debug/admin menu** to adjust values, an **(extra) keyframe editor** if time allows, and a **JSON serializer**. Design features: the **stage/arena determines available environmental items**; a **roster of ≥6 unique characters**; the **gesture input system** simplifying combos/specials; and a **wide variety of interactable items**. Team: **6 CS + 1 Design + 2 Artists**.

Note (module completeness): the proposal doesn't mention an **External Database** or accounts, but the module mandates Firebase — plan to persist unlocked characters / cosmetics / match records there (a standard, small addition; see the External Database row + M1G12-style integration). *Monetization:* none is specified — if added, keep it **cosmetic / non-pay-to-win** (no selling characters/power that affect the fight) per the module's clean-monetization rule.

References:

- Super Smash Bros. Ultimate — Wikipedia (the benchmark fighter built around **items and stage hazards as weapons** — fighters grab, throw and weaponise the environment on a wide side-view stage — the environment-is-the-weapon precedent)
- Brawlhalla on Steam (a free-to-play platform fighter with a large roster of unique "Legends" and **weapon pickups**, running cross-platform including mobile — the roster-of-distinct-fighters + pick-up-a-weapon + clean F2P precedent)
- Shadow Fight 3 — Google Play (a popular **mobile** martial-arts fighting game with a **simplified touch-control scheme** and distinct fighting styles/factions — the mobile + simple-touch-combat precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Kungfu Crashout
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — the game is explicitly a native Android build with gesture (no-button) touch controls. The Native PC build is the development/tools vehicle (the hurtbox/frame-debug editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts, unlocked-character/cosmetic persistence and match records; peer-to-peer is the in-match transport.

Front end: A 2D side-view fighter renderer — the two characters (animated with frame data), the detailed interactable stage, the pick-up/throwable items, hit-spark VFX and particles, and the minimal gesture-driven HUD.

Languages: Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The deterministic real-time C++ fight engine — the fixed-timestep loop holding 60 fps with 2 characters + physics objects + particles, hitbox/hurtbox collision + frame data, the grab/throw/environmental-object physics, the gesture-to-action recognizer, and the enemy AIs — plus a peer-to-peer netcode layer for PvP; Firebase for accounts/persistence on the Extended tier.

Languages: C++ (engine + netcode); Firebase-backed services on the Extended tier.

Backend scope: Local + P2P now, server-ready (T2) — the accounts/persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the deterministic C++ loop is the heart — it advances both characters, resolves hitbox/hurtbox collisions on frame data, simulates the throwable physics objects + particles, and holds 60 fps on the Native-Android target.
- *Networking:* **core feature** — peer-to-peer PvP. Note this is the hardest part of a fighting game: a deterministic lockstep/**rollback** netcode over P2P is a serious undertaking, so the local 2-player + AI fight is the reliable floor and networked PvP is the primary scope-cut candidate (see scope note).
Application Communication ✓ covers accounts/persistence.
- *Data-driven:* characters + movesets/frame data, the ≥ 6 roster, the item catalogue and per-stage item availability are all data — a new character, move or item is a data change (authored/tuned via the editor).
- *2D:* the side-view fighter renderer, the detailed interactable stage, throwable items, hit-spark VFX and particles (custom-rendered). *3D:* not needed.
- *Custom tool:* the **fighting-game debug editor** — hurtbox/hitbox visualization, frame checking of movement + animation, a value-tuning admin menu, a JSON serializer, and (time permitting) a keyframe editor.
- *External DB:* accounts, unlocked characters/cosmetics, match records (Firebase — add this per the module). *Auth » / Analytics »:* module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the deterministic 60 fps fight loop + the **gesture** recognizer.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a deterministic fixed-timestep C++ + loop advancing 2 characters + physics objects + particles at 60 fps, with a gesture-input recognizer mapping strokes to movement/attacks/interactions (no on-screen buttons); C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; physics (M1G5) and animation (M1G9) run on its tick; netcode (M2G7) needs its determinism.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An entity/component model with characters + movesets/frame data, the ≥6 roster, the item catalogue and per-stage item availability in hot-reloadable JSON — a new character, move or item is a data change (the serializer round-trips it).
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the debug editor), M2G10 (AI data), M2G11 (item/throw physics data).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — the base of the hurtbox/frame debug tooling.

Tech & dependencies

- **Tech:** ImGui overlays drawing hitboxes/hurtboxes + frame data, a frame-step/pause debugger, Tracy profiling to prove 60 fps with 2 characters + physics + particles, Catch2 tests (determinism, collision, data loading), on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G2 (the full debug editor); M2G3 (CI hardening).
- **G4 2D Graphics** — the side-view fighter renderer + detailed interactable stage + particles.

Tech & dependencies

- **Tech: Web build:** A sprite renderer for the 2 characters, the wide detailed stage with interactable elements, throwable items, hit-spark VFX and particles, with a camera and the minimal HUD; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4-style polish; every visible frame; the debug editor shares this renderer.
- **G5 Physics & Collision** — hitbox/hurtbox collision + the throwable physics objects.

Tech & dependencies

- **Tech:** A C++ collision system (broadphase + resolution) for character hit/hurt boxes on frame data and for the grabbable/throwable physics objects that interact with characters and stage; GLM math, deterministic on both clients.
- **Prerequisites:** G1 (loop & tick), G2 (move/item data).
- **Feeds into:** M2G11 (advanced throw/grab + continuous collision); the environment-is-the-weapon pillar.
- **G9 Animation System** — character animation + frame data (the combat backbone).

Tech & dependencies

- **Tech:** A sprite-animation system with a state graph and per-frame data (startup/active/recovery, hit/hurt boxes bound to frames), blending and cancels, driven by move data — the substrate the frame-checking tool inspects.
- **Prerequisites:** G1 (loop), G4 (rendering).
- **Feeds into:** The debug editor's frame checking; the game feel of every move.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — hold 60 fps on-device + the persistence seam.

Tech & dependencies

- **Tech: Web build:** Memory pools for physics/particles, Tracy profiling of a busy fight, ASan on MSVC, and on-device validation (Pixel 9a) holding 60 fps with 2 characters + physics + particles; the Firebase/Linux-server seam (accounts/persistence) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).
 - **Feeds into:** The Native-Android deliverable + a stable 60 fps fight.
- **G2 ★ Content Editor — Core (Fighting-Game Debug Editor)** — hurtbox/hitbox + frame checking + admin menu + serializer.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: visualize + edit hit/hurt boxes per frame, step frames to check movement/animation, a value-tuning admin menu, and JSON serialize/deserialize of characters/moves/items; (stretch) a keyframe editor.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** M1G3 (debug overlays), M1G9 (animation/frame data), G2 (data model).
- **Feeds into:** Fast move/character authoring + balancing; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (determinism, collision/frame data, netcode sync under simulated latency, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.
- **G7 Networking — Match Sync & Prediction (P2P PvP + Rollback)** — the peer-to-peer fighting netcode.

Tech & dependencies

 - **Tech:** A P2P transport + a deterministic lockstep/rollback model (input exchange, prediction, rollback + re-simulate on misprediction), state-hash desync detection, tested under simulated latency/jitter/loss; leans on the M1 deterministic loop. *Highest-risk goal — see scope note.*
 - **Prerequisites:** M1G1 (deterministic loop), M1G3 (desync detection).
 - **Feeds into:** Online PvP at showcase (stretch); the local 2-player fight is the floor.
- **G10 AI — Advanced (Difficulty & Character-Based Enemy AIs)** — the different enemy AIs.

Tech & dependencies

 - **Tech:** Behaviour trees / FSMs giving distinct per-character AI (spacing, punishes, item usage), difficulty tiers (reaction time, decision quality), and use of the environment/items, tuned from data (custom C++).
 - **Prerequisites:** M1G1 (fight state), G2 (move/character data).
 - **Feeds into:** The single-player + practice modes; the "different enemy AIs" feature.
- **G11 Physics — Advanced (Grab / Throw / Environmental Interaction)** — the environment-as-weapon systems.

Tech & dependencies

 - **Tech:** Constraint/joint solvers + continuous collision for grabbing, carrying and throwing stage objects at the opponent (impact damage/knockback), interaction rules per item type, and per-stage item availability; deterministic on both clients.
 - **Prerequisites:** M1G5 (physics & collision).
 - **Feeds into:** The environment-is-the-weapon USP; the stage-determines-items design pillar.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — **as proposed**, and well-justified: a fighting game is **art- and animation-heavy** (a ≥ 6 -character roster each with a distinct style and a full move-set of animations, detailed interactable stages, throwable items, hit VFX), so **2 artists** are the right call, one able to specialise in character animation (fighting games live on frame-by-frame animation). A single designer owns the FDD, the **gesture control scheme + game feel**, the roster/item/stage systems and the AI/difficulty design (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). *Scope note (advisory):* a deterministic 60 fps fight engine + frame-data animation + hitbox/hurtbox combat + grab/throw physics + a debug editor is already a full CS scope for M1/M2 — and **peer-to-peer rollback netcode is, on its own, one of the hardest problems in games**. Strong recommendation: make the **local 2-player + AI fight the floor** (fully playable, great feel, ≥ 3 characters, one or two stages), and treat **networked P2P PvP as the stretch** (M2G7) — do a networking spike early to de-risk it, and keep the sim deterministic from day one so rollback stays possible. The ≥ 6 roster, the full item variety and online PvP are the M2 stretch, not the floor.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the environment-is-the-weapon fighter, the gesture control scheme, the ≥ 6 roster + distinct styles, the stage-determines-items design; primary benchmark e.g. Super Smash Bros. / Brawlhalla, plus a Singapore benchmark)
- M1D2 Core Gameplay Loop Prototype (a greybox proving the gesture combat + grab-and-throw-the-environment fight is fun) · prototype in Unity
- M1D4 Game Feel & Touch Interaction Specification (the no-button gesture scheme — how movement/attacks/grabs/specials map to strokes, buffering, cancels, hit-stop — as tunable data)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D3 Systems Balance & Tuning — Data-Driven (character/move/item balance and AI difficulty via the editor's JSON, no code changes — critical for a fighter)
- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the health/round HUD, the gesture affordances/feedback and the character-select, plus a wireframe baseline.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — character-select, match HUD, results and the (local/online) match-setup navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished gesture-combat UX tested with players on the Pixel 9a (does the no-button scheme actually read?).
- **Art M1:** 2 artists justified (≥ 6 fighters + animations + stages + items + VFX)
 - M1A1 ★ Art Bible / Proof of Concept (the martial-arts visual direction — tone, palette, character-identity language)
 - M1A2 Character Concepts & Model Sheets (the ≥ 6 fighters with distinct styles + silhouettes)
 - M1A3 Environment, Props & VFX Concepts (the detailed interactable stages, the throwable items, hit-spark VFX)
 - M1A4 UI Art Concepts & Typography (health/round HUD, character-select, gesture feedback)
 - M1A5 Animation Foundations (fighting key poses, attack/hurt/throw cycles — the fighter's lifeblood)
 - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (full move-sets, throws, environmental grabs, KOs across the roster)
 - M2A3 Environment Alive — Ambient Animation & Layering (living, interactable stages)
 - M2A4 UI Art — States & Feedback (hit/combo/round feedback states)

- M2A5 VFX — Critical Effects & Transitions (impacts, item-throw effects, specials)
- M2A6 Cutscenes & Storyboards (character intros / victory poses)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (stage BGM, hit/impact, whiff, item-pickup/throw, KO cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping fight SFX.
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Games floor (stage BGM + hit / block / throw / special / KO / announcer SFX, ≥6 types), scaling with fight intensity.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct hit/throw/special cues).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the hurtbox/frame debug editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*